

Heavy-Light Decomposition

Sean

October 4, 2026

Outline

- Heavy edges and chains
- Flattening the tree
- Answering path queries
- C++ implementation and extensions

Example: Path Queries II

CSES: Path Queries II

Each vertex has a value. Support changing one value and finding the maximum value on a path between any two vertices.

- $1 \leq n, q \leq 2 \cdot 10^5$
- The tree's edges stay fixed as values change
- Include both endpoints of the path



Tree paths and array queries

- A segment tree handles an interval in an array
- Euler tour order makes a subtree one interval
- A path between arbitrary vertices can cross several subtrees

Heavy-light decomposition (HLD)

Split the tree into chains so any path becomes $O(\log n)$ array intervals.



1 Heavy edges and chains

2 Flattening the tree

3 Answering path queries

4 Implementation and extensions

Heavy edges and chains

What makes an edge heavy?

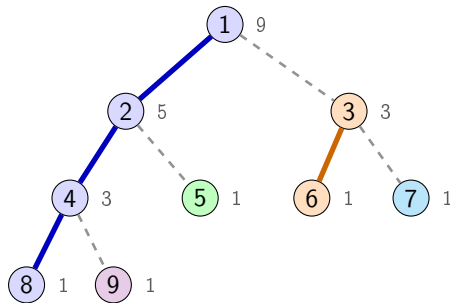
Root the tree at 1 and compute subtree sizes $\text{sz}[u]$.

- For each non-leaf, choose a child with the largest subtree
- The edge to that child is **heavy**
- Edges to the other children are **light**
- Break ties in any consistent way

A leaf has no heavy child. “Heavy” refers to subtree size, not an edge weight.



Choosing heavy children

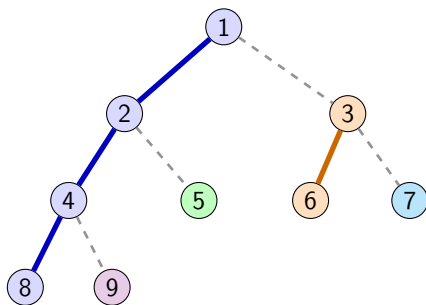


Numbers beside vertices are subtree sizes.

- At 1: choose 2 ($5 > 3$)
- At 2: choose 4 ($3 > 1$)
- At 3 and 4: break ties

Solid edges are heavy. Dashed edges are light.

Heavy edges form chains



Remove the light edges. The chains are:

- 1, 2, 4, 8
- 3, 6
- 5
- 9
- 7

Each vertex has at most one heavy child. A chain may contain a single vertex.

A light edge halves the subtree

Let c be a light child and h the heavy child of u .

- The chosen child has $\text{sz}[h] \geq \text{sz}[c]$
- The two child subtrees are disjoint

$$\text{sz}[u] \geq 1 + \text{sz}[h] + \text{sz}[c] \geq 1 + 2 \text{sz}[c]$$

Every downward light edge

$$\text{sz}[c] < \frac{\text{sz}[u]}{2}$$



Why a path crosses few chains

- Each downward light edge cuts the subtree size by more than half
- A root-to-vertex path crosses $O(\log n)$ light edges
- Each light edge starts a new chain

An arbitrary path

Split the path at its LCA. Each side crosses $O(\log n)$ chains, so the whole path does too.

The largest child need not contain half the subtree. The bound is on light children.



- 1 Heavy edges and chains
- 2 Flattening the tree
- 3 Answering path queries
- 4 Implementation and extensions

Flattening the tree

Chain heads and array positions

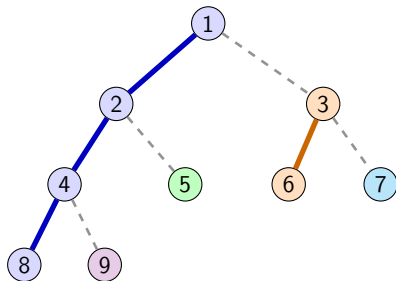
- $\text{head}[u]$: the highest vertex in u 's chain
- $\text{pos}[u]$: the index assigned to u in one global array
- Visit each vertex, then its heavy child, then its light children

Heavy-first DFS order

This traversal puts every heavy chain in consecutive array positions, from top to bottom.



One tree becomes one array



Finish the heavy chain first, then visit the pending light subtrees.
One valid order:

1, 2, 4, 8, 9, 5, 3, 6, 7

1	2	4	8	9	5	3	6	7
0	1	2	3	4	5	6	7	8

The arrays for our example

Vertex u	1	2	3	4	5	6	7	8	9
head[u]	1	1	3	1	5	3	7	1	9
pos[u]	0	1	6	2	5	7	8	3	4

- Vertices 1, 2, 4, 8 share head 1 and positions 0, 1, 2, 3
- Vertex 5 starts its own chain, so head[5] = 5
- Store vertex values using $A[\text{pos}[u]] = \text{value}[u]$



A path inside one chain

If u and v share a chain, assume u is shallower.

$$\text{path } u \text{ to } v \iff [\text{pos}[u], \text{pos}[v] + 1)$$

1	2	4	8
0	1	2	3

Path 2 to 8 uses positions $[1, 4)$, including both endpoints.
Throughout this lecture, $[l, r)$ includes l and excludes r .



Subtrees remain consecutive too

Heavy-first order is still a DFS preorder.

$$\text{subtree of } u \quad \longleftrightarrow \quad [\text{pos}[u], \text{pos}[u] + \text{sz}[u])$$

- For vertex 2: positions $[1, 6)$ hold 2, 4, 8, 9, 5
- For vertex 3: positions $[6, 9)$ hold 3, 6, 7
- One segment tree can support both subtree and path queries

- 1 Heavy edges and chains
- 2 Flattening the tree
- 3 Answering path queries
- 4 Implementation and extensions

Answering path queries

Removing one chain segment

While the endpoints have different chain heads:

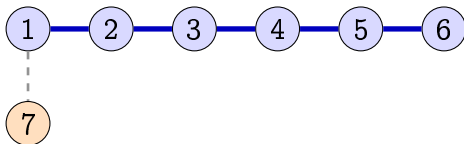
- 1 Choose the endpoint u whose *chain head* is deeper
- 2 Query from $\text{head}[u]$ to u , inclusive
- 3 Replace u with $\text{parent}[\text{head}[u]]$

Each step crosses one light edge

The queried segment lies entirely on the remaining path. Its head's parent is the next vertex on that path.



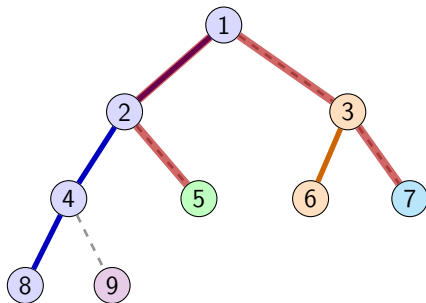
Why compare the chain heads?



Query the path from 6 to 7, with root 1.

- Vertex 6 is deeper, but its chain starts at root 1
- Vertex 7 has the deeper chain head, so remove 7 first
- Removing 6's whole chain first would jump above the root

Example: the path from 5 to 7



The path is 5, 2, 1, 3, 7.
It uses four chain segments:

- 5
- 7
- 3
- 1, 2

We may process these pieces in any order when combining maxima.

A complete path-query trace

Let vertex values for $1, \dots, 9$ be $5, 3, 7, 6, 2, 4, 8, 1, 9$.

Endpoints	Piece removed	Array interval	Maximum
5, 7	5	$[5, 6)$	2
2, 7	7	$[8, 9)$	8
2, 3	3	$[6, 7)$	7
2, 1	1, 2	$[0, 2)$	5

$$\text{pathMax}(5, 7) = \max(2, 8, 7, 5) = 8$$

The first two chain heads have equal depth. Either endpoint may go first.



The final segment and the LCA

Once the endpoints share a chain:

- The shallower endpoint is the LCA of the original pair
- Query the interval between the two endpoints, including both
- Combine it with the pieces already processed

Why no vertex is counted twice

After removing a segment through its head, continue at the head's parent. The final segment includes the LCA exactly once.

The chain loop finds the split itself. A separate LCA query is optional.



Updating a vertex value

To assign $\text{value}[u] = x$, update the array at $\text{pos}[u]$.

Operation	Array action	Result
<code>pathMax(5, 7)</code>	Four intervals	8
Set vertex 1 to 10	Assign $A[0] = 10$	
<code>pathMax(5, 7)</code>	Same four intervals	10

- Only the stored values change
- The parent, heavy child, head, and position arrays stay valid



Complexity with a segment tree

Operation	Cost
Build HLD and the segment tree	$O(n)$
Assign one vertex value	$O(\log n)$
Query one array interval	$O(\log n)$
Query an arbitrary tree path	$O(\log^2 n)$
Total memory	$O(n)$

A path has $O(\log n)$ chain segments. Each segment takes one range query.



- 1 Heavy edges and chains
- 2 Flattening the tree
- 3 Answering path queries
- 4 Implementation and extensions

Implementation and extensions

C++ setup and conventions

- C++17, with standard headers and using namespace std;
- Read n, q , value[1.. n], and the undirected adjacency list g
- Vertices use 1 to n . Array positions use 0 to $n - 1$
- Use root 1 and vertex 0 as the missing-parent/child sentinel
- The following snippets belong in one scope after reading the tree

All passes are iterative, including for a chain of 200 000 vertices.



C++: parents, depths, and order

```
vector<int> parent(n + 1, 0), depth(n + 1, 0);
vector<int> order{1};
for (int i = 0; i < (int)order.size(); ++i) {
    int u = order[i];
    for (int v : g[u]) if (v != parent[u]) {
        parent[v] = u;
        depth[v] = depth[u] + 1;
        order.push_back(v);
    }
}
```

The graph is a tree. Skipping the parent visits every other vertex exactly once.



C++: subtree sizes and heavy children

```
vector<int> sz(n + 1, 1), heavy(n + 1, 0);
for (int i = n - 1; i >= 0; --i) {
    int u = order[i];
    for (int v : g[u]) if (parent[v] == u) {
        sz[u] += sz[v];
        if (heavy[u] == 0 || sz[v] > sz[heavy[u]])
            heavy[u] = v;
    }
}
```

Children finish first in reverse order. Ties keep the first largest child in the adjacency list.



C++: heavy-first positions

```
vector<int> head(n + 1), pos(n + 1);
vector<pair<int, int>> todo{{1, 1}};
int timer = 0;
while (!todo.empty()) {
    auto [u, h] = todo.back(); todo.pop_back();
    head[u] = h; pos[u] = timer++;
    for (int v : g[u])
        if (parent[v] == u && v != heavy[u])
            todo.push_back({v, v});
    if (heavy[u]) todo.push_back({heavy[u], h});
}
```

This is a stack. Push the heavy child last so it is processed first.



C++: one maximum segment tree

```
const int NEG = numeric_limits<int>::lowest();
int base = 1;
while (base < n) base *= 2;
vector<int> seg(2 * base, NEG);
for (int u = 1; u <= n; ++u)
    seg[base + pos[u]] = value[u];
for (int i = base - 1; i > 0; --i)
    seg[i] = max(seg[2 * i], seg[2 * i + 1]);
```

- Leaves store the values in HLD order
- NEG is the identity for maximum, including negative values



C++: point assignment

```
auto assign = [&](int p, int x) {  
    p += base;  
    seg[p] = x;  
    while (p > 1) {  
        p /= 2;  
        seg[p] = max(seg[2 * p], seg[2 * p + 1]);  
    }  
};
```

To change vertex u , call `assign(pos[u], x)`. The argument p is an array position, not a vertex number.



C++: maximum on an interval

This helper queries $[l, r)$.

```
auto rangeMax = [&](int l, int r) {  
    int result = NEG;  
    for (l += base, r += base; l < r; l /= 2, r /= 2) {  
        if (l & 1) result = max(result, seg[l++]);  
        if (r & 1) result = max(result, seg[--r]);  
    }  
    return result;  
};
```

Empty intervals return NEG. Both indices refer to the global HLD array.



C++: maximum on a tree path

```
auto pathMax = [&](int u, int v) {  
    int result = NEG;  
    while (head[u] != head[v]) {  
        if (depth[head[u]] < depth[head[v]]) swap(u, v);  
        int h = head[u];  
        result = max(result, rangeMax(pos[h], pos[u] + 1));  
        u = parent[h];  
    }  
    if (depth[u] > depth[v]) swap(u, v);  
    return max(result, rangeMax(pos[u], pos[v] + 1));  
};
```

The final call includes the LCA and handles $u = v$ as a one-vertex path.



C++: processing the queries

```
while (q-- > 0) {  
    int type, a, b;  
    cin >> type >> a >> b;  
    if (type == 1) assign(pos[a], b);  
    else cout << pathMax(a, b) << '\n';  
}
```

The segment tree holds the current values. Preprocessing runs once before this loop.



Extension: values on edges

Store edge $(\text{parent}[v], v)$ at $\text{pos}[v]$.

- The root's position stores the identity
- A cross-chain query still includes the chain head's position, which stores the incoming light edge
- In the final common chain, skip the shallower endpoint's position

$$[\text{pos}[u] + 1, \text{pos}[v] + 1) \quad (u \text{ shallower than } v)$$

An edge path from a vertex to itself is empty. Handle its result as the problem requires.



Other path operations

- Sum, minimum, gcd, and xor use the same chain decomposition with a suitable segment tree
- Choose the matching identity and an integer type wide enough for the answer
- Path updates can use lazy propagation on the same intervals

When order matters

String concatenation and matrix multiplication need forward/reverse aggregates and separate accumulators for the two endpoints.



Common mistakes

- Comparing endpoint depths instead of chain-head depths
- Jumping to `parent[u]` instead of `parent[head[u]]`
- Mixing vertex numbers with array positions
- Forgetting the $+1$ at the right end of a half-open interval
- Counting the LCA as an edge in an edge-value query



Practice roadmap

- CSES: Path Queries II
Point assignments and maximum on arbitrary paths
- CSES: Subtree Queries
Apply the subtree interval and a sum segment tree
- CSES: Path Queries
Sum on paths from the root to a vertex

The last two also have simpler Euler-tour solutions. Use them to compare the approaches.



HLD in one pass through the idea

- 1 Choose the largest child at each vertex
- 2 Visit heavy children first to make chains consecutive
- 3 Store the flattened values in one segment tree
- 4 Query the deeper chain head's segment, then jump above it
- 5 Finish with the interval inside the shared chain

Costs

$O(n)$ preprocessing, $O(\log n)$ point updates, and $O(\log^2 n)$ path queries.



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

